Mineração de Métricas DORA em Repositórios Open-Source que Utilizam GitHub Actions

Pedro Maia Alves[1], **Diogo Brunoro**[1], **Lorran Avelar**[1]

[1]Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Abstract. *This study investigates delivery performance in popular open-source repositories that use GitHub Actions. Releases, commits, and workflow runs are adopted as explicit operational proxies for the four DORA metrics. The study also evaluates how alternative definitions affect repository classification. This document presents the research questions, hypotheses, collection protocol, and the frozen sample of 300 repositories. Independent manual validation and heuristic qualification are still pending.*

Resumo. *Este estudo investiga o desempenho de entrega em repositórios open-source populares que utilizam GitHub Actions. Releases, commits e execuções de workflows são adotados como proxies operacionais explícitos para as quatro métricas DORA. O estudo também avalia como definições alternativas afetam a classificação dos repositórios. Este documento apresenta as questões de pesquisa, hipóteses, protocolo de coleta e a amostra congelada de 300 repositórios. A validação manual independente e a qualificação da heurística ainda estão pendentes.*

1. Introdução

As métricas DORA (*DevOps Research and Assessment*) tornaram-se o padrão de mercado para medir o desempenho de entrega de software [Forsgren et al. 2018, DORA sd]. Quatro métricas descrevem duas dimensões: velocidade, com a frequência de implantação (*deployment frequency*) e o tempo de entrega de mudanças (*lead time for changes*); e estabilidade, com a taxa de falha das mudanças (*change failure rate*, CFR) e o tempo de recuperação.

Medir essas métricas em projetos *open-source* esbarra em um problema de construto: o GitHub não registra implantações em produção nem falhas em produção. O que existe são *releases*, *commits* e execuções de *workflows* de integração contínua. Neste trabalho, esses artefatos funcionam como *proxies*: uma *release* publicada representa uma implantação, e uma execução de CI que falhou representa uma falha. Um *proxy* pode enganar. Uma biblioteca que publica uma *release* por mês não implanta nada em produção, pois quem faz isso são os seus usuários, e uma falha de pipeline não é uma falha em produção.

O objetivo deste estudo é minerar automaticamente as métricas DORA em repositórios populares do GitHub que usam GitHub Actions e avaliar quanto se pode confiar nessas medidas. Para isso, o grupo (i) calcula as métricas com definições operacionais explícitas, (ii) valida os critérios automáticos contra o julgamento humano e (iii) mede o quanto as conclusões mudam quando a definição da métrica muda. Este artigo responde às seguintes questões de pesquisa (RQ):

- **RQ01:** qual a frequência de *deploys* dos repositórios populares que usam CI/CD?
- **RQ02:** qual o tempo entre um *commit* e o seu respectivo *deploy*?
- **RQ03:** qual a taxa de falha das mudanças entregues por esses repositórios?
- **RQ04:** qual o tempo de recuperação após uma execução de CI/CD com falha?
- **RQ05:** repositórios com maior frequência de *deploy* apresentam maior ou menor taxa de falha?
- **RQ06:** quais características dos repositórios estão associadas a um melhor desempenho DORA?
- **RQ07:** o quanto a classificação DORA de um repositório depende da definição operacional escolhida?

1.1. Hipóteses

As hipóteses abaixo foram escritas antes de qualquer análise dos dados coletados.

- **H1 (RQ01).** Esperamos que a mediana fique abaixo de uma *release* por semana, pois a maioria dos projetos populares são bibliotecas e ferramentas que publicam versões por ciclos de funcionalidades, e não de forma contínua. Esperamos também uma cauda longa, com poucos repositórios publicando *releases* diariamente.
- **H2 (RQ02).** Esperamos que o *lead time* por *release* (variante a) seja bem maior que o por *commit* (variante b), porque a variante (a) é dominada pelo *commit* mais antigo de cada *release*, enquanto a (b) pondera todos os *commits*, a maioria deles escrita perto da publicação.
- **H3 (RQ03).** Esperamos que o CFR de pipeline (variante a) seja maior que o CFR de entrega (variante b), pois falhas de CI são baratas, frequentes e ocorrem justamente para impedir que o problema chegue a uma *release*, enquanto *releases* corretivas em até sete dias são mais raras.
- **H4 (RQ04).** Esperamos que a mediana do tempo de recuperação seja de poucas horas, já que falhas de CI costumam ser corrigidas por um novo *commit* no mesmo dia, mas com cauda pesada e uma proporção não desprezível de episódios censurados, de *workflows* quebrados que ficam abandonados por semanas.
- **H5 (RQ05).** Esperamos uma correlação fraca e negativa entre a frequência de *deploys* e o CFR, isto é, que repositórios que entregam com maior frequência tenham taxa de falha igual ou menor. Essa expectativa acompanha a evidência do DORA de que velocidade e estabilidade não constituem necessariamente um *trade-off* [DORA 2026]. Entretanto, a associação pode ser mais fraca nos nossos proxies, já que falhas de CI não representam necessariamente falhas das versões entregues, e correlação não demonstra causalidade.
- **H6 (RQ06).** Esperamos que o tipo do projeto e o tamanho da comunidade de contribuidores apresentem associações mais claras com o desempenho DORA do que a linguagem principal. Aplicações e serviços devem publicar com maior frequência, enquanto comunidades maiores podem sustentar revisão, manutenção e recuperação mais contínuas. Popularidade, idade e linguagem podem apresentar associações menos uniformes; elas serão interpretadas sem atribuição de efeito causal.
- **H7 (RQ07).** Esperamos concordância moderada ou alta entre as classificações, mas com mudanças nos repositórios próximos aos limites das categorias. Incluir pré-releases ou tags pode aumentar a frequência medida; trocar o *lead time* por

release pelo *lead time* por commit altera a ponderação; e substituir falhas de CI por releases corretivas observa outro tipo de falha. Essa sensibilidade reforça a necessidade de explicitar os proxies usados.

2. Metodologia

2.1. Desenho e população

O estudo é observacional e utiliza dados públicos do GitHub. A janela oficial é de 1 de outubro de 2025 a 30 de setembro de 2026. A amostra completa deve conter pelo menos 300 repositórios não arquivados e não derivados de forks, com GitHub Actions, no mínimo cinco releases principais e 50 execuções válidas de CI na janela. Os candidatos são embaralhados com semente 42 e avaliados em ordem até atingir a meta. O funil diferencia a população retornada pela busca dos candidatos efetivamente avaliados. O snapshot reutilizado mantém as consultas de origem; faixas adicionais previstas no config não são apresentadas como consultas que já ocorreram.

Foram retornados 53.143 candidatos e avaliados 2.644, chegando a 300 repositórios. As consultas do snapshot abrangem 1.000 a 8.000 estrelas.

Tabela 1. Funil de seleção dos repositórios.

Etapa	Entraram	Descartados	Restaram
Candidatos retornados pela busca	53.143	–	53.143
Candidatos avaliados (ordem aleatória)	2.644	–	2.644
Usam GitHub Actions	2.644	1.149	1.495
≥ 5 releases publicadas na janela	1.495	1.082	413
≥ 50 execuções de CI válidas	413	113	300
Amostra final	–	–	300

2.2. Definições operacionais

CI corresponde às execuções disparadas por `push` no default branch. Sucesso é a conclusão `success`; falha inclui `failure`, `timed_out` e `startup_failure`. Os demais estados são ignorados e contabilizados separadamente. Releases não draft e não pré-release, publicadas na janela, representam entregas; essa aproximação não comprova implantação em produção.

A frequência de entrega é o número de releases principais dividido pela duração inclusiva da janela em semanas. As variantes incluem pré-releases ou utilizam tags, cuja data é a autoria do commit apontado, e não uma data de criação da tag. O lead time utiliza `commit.author.date`: na variante (a), calcula-se a mediana do maior tempo de cada release; na variante (b), a mediana das relações release/commit válidas. Ambas são expressas em horas. Commits inválidos, comparações indisponíveis e releases sem commits não são convertidos em tempos iguais a zero.

O CFR (a) é a quantidade de runs com falha dividida por falhas mais sucessos. O CFR (b) identifica uma release com falha quando outra release corretiva é publicada posteriormente, em até sete dias, inclusive. As sete últimas datas da janela ficam fora do denominador por censura à direita. O tempo de recuperação é medido por workflow,

do início da primeira falha após um sucesso ao término do próximo sucesso; falhas consecutivas pertencem ao mesmo episódio. Episódios sem sucesso posterior permanecem censurados, e falhas iniciais sem sucesso anterior não abrem um episódio observável.

2.3. Amostra-ouro e validação da heurística

O sorteio canônico usa `validacao.amostra_ouro`: a população é ordenada por nome antes de selecionar 60 repositórios com semente 42; as releases principais de cada repositório são ordenadas por identificador antes de selecionar cinco, também com semente 42. A população de origem é a amostra completa congelada, não o piloto de 100 repositórios. As três planilhas têm os mesmos identificadores e links, sem previsões da heurística.

O protocolo prevê três avaliações independentes do tipo do projeto, da existência de entregas reais e do objetivo corretivo de cada release. Biblioteca/framework, aplicação/serviço, ferramenta CLI e outro são as categorias de tipo. Correção de defeito recente, regressão, vulnerabilidade ou reversão defeituosa deve ser o objetivo principal de uma release corretiva; versões mistas sem esse foco são negativas. O registro distingue avaliação humana de análise assistida por IA. Uma análise única não equivale a três avaliações independentes.

A concordância é calculada pelo kappa de Fleiss, com interpretação descritiva de Landis–Koch [Landis and Koch 1977]. O consenso usa unanimidade ou maioria simples; os casos 1–1–1 exigem decisão adicional identificada e justificada, sem declarar uma reunião que não ocorreu. A origem, os hashes e a cobertura dos rótulos devem corresponder à amostra-ouro antes de incorporar o consenso ao dataset.

A heurística v1 combina patch SemVer crescente, mantendo major/minor, com a presença dos tokens completos `fix`, `hotfix` ou `revert` nos commits. Tags não-SemVer usam apenas o critério textual. Precisão, recall e F1 são calculados contra o consenso da amostra definitiva. Cada regra avaliada fica registrada, inclusive quando não atinge F1 de 0,70. Resultados obtidos sobre a amostra piloto não certificam automaticamente a nova população.

A validação definitiva ainda depende das avaliações independentes da nova amostra; os kappas e o F1 do piloto não são apresentados como resultados finais da S02.

2.4. Ameaças à validade e reprodução

Releases, workflows e recuperação de CI são proxies e podem divergir de entregas e incidentes em produção. Notas incompletas, tags móveis, versões paralelas de monorepositórios e ambiguidades de classificação limitam a interpretação. A censura e as informações ausentes são quantificadas, não substituídas por zeros. Refinar e avaliar uma heurística na mesma amostra pode superestimar sua generalização; o limiar de F1 não constitui evidência de desempenho em dados não vistos. Se forem usados avaliadores de IA, devem ser declarados os modelos, a separação entre análises e a ausência de três avaliações humanas; agentes do mesmo modelo também podem compartilhar vieses.

Código, testes, dados e manifestos para reprodução:

`https://github.com/PedroMaiaAlves/lab-experimentacao-software`

2.5. Coleta completa e consolidação do dataset

A coleta da Sprint 2 utiliza a janela de 1 de outubro de 2025 a 30 de setembro de 2026, semente 42 e meta mínima de 300 repositórios após os filtros de inclusão. O comando de coleta permite reaproveitar um snapshot versionado de candidatos; nesse caso, sua ordem e consultas de origem são preservadas. Uma nova busca pode ampliar as faixas de estrelas, com subdivisão para respeitar o teto da Search API. Não se considera a amostra final congelada antes da coleta e validação completas.

O pipeline utiliza Python e `requests`, sem bibliotecas que encapsulem a API do GitHub. São consultados os endpoints de workflows, releases, comparações entre tags, commits, contribuidores e execuções do GitHub Actions. As tags podem ser recuperadas em lotes por GraphQL. Releases e comparações são paginadas; a data de autoria do commit é utilizada nos cálculos de lead time e nas datas das tags. Os runs são restritos ao default branch, ao evento push e à janela do estudo. Consultas mensais com pelo menos mil resultados são subdivididas; intervalos diários ainda saturados são registrados como incompletos e impedem o congelamento.

O cache em disco permite reutilizar respostas, enquanto checkpoints registram o avanço da seleção e as coletas completas por repositório. O cliente respeita os limites indicados nos cabeçalhos e aplica novas tentativas com espera crescente em falhas temporárias. Erros transitórios não constituem evidência de exclusão. Comparações com erro 404 são contabilizadas e excluídas do lead time; datas inválidas de commits também são registradas e excluídas das medianas.

A execução completa terminou em 15 de outubro de 2026, às 05:15 UTC. O ambiente de execução utilizou Python 3.12.14. A instrumentação parcial, iniciada após o começo da coleta, registrou 25.634 respostas HTTP (25.624 com status 200 e dez com status 404) e três esperas, somando três segundos solicitados. Esses valores não incluem acertos de cache nem representam todas as requisições desde a seleção inicial. As esperas registradas não comprovam, isoladamente, ocorrência de rate limit. As 311 fontes congeladas são distribuídas em arquivo ZIP, com verificação SHA-256; a restauração e o recálculo sem acesso à API reproduziram exatamente o dataset.

O dataset reutiliza as funções de cálculo do grupo, mantém uma linha por repositório e registra métricas, versão da heurística e contagens de ignorados e censurados. O CFR de CI considera falhas sobre falhas mais sucessos. O CFR por release observa uma corretiva posterior em até sete dias, excluindo do denominador as releases das sete últimas datas da janela. Episódios de recuperação não encerrados são mantidos como censurados. Valores ausentes não são substituídos por zero, nem tipos de projeto são inferidos automaticamente fora do consenso. O consenso somente é incorporado quando seu manifesto confirma a origem da rotulagem e o vínculo, por hashes, com a amostra congelada. O consenso legado sem esse vínculo não é considerado validação da nova coleta. A revisão de C é assistida por IA e provisória sobre a amostra anterior; não equivale a uma avaliação humana nem substitui o fechamento da amostra-ouro pelos três avaliadores.

A execução concluída reuniu 300 repositórios, na janela de 2025-10-01 a 2026-09-30. Foram contabilizados 11453 releases principais, 291405 runs válidos e 22871 runs ignorados. Houve 9 comparações com erro 404 e 561 relações release/commit com data inválida. No CFR (b), 274 releases foram censuradas. Dos 7861 episódios de falha, 90

permaneceram censurados. Foram registrados 56 intervalos saturados (incluindo subdivisões), com 0 incompletos. O tipo do projeto permanece ausente em 300 repositórios não cobertos pelo consenso, sem imputação automática. O CFR (b) ficou indisponível em 7 repositórios por evidência textual insuficiente.

Referências

DORA (2026). DORA's software delivery performance metrics. `https://dora.dev/guides/dora-metrics/`. Acesso em: 7 out. 2026.

DORA (s.d.). Accelerate state of DevOps reports. `https://dora.dev/research/`. Acesso em: out. 2026.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.

Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.